Sending embeddings from 10 edge nodes to one node with NebulaStream

Demo report · LLMs in Hierarchical Aggregation Topologies · Vasil · 11 November 2026

1. What was run

10 NebulaStream edge workers (A1–A10) each read a slice of the 20 Newsgroups dataset that was turned into sentence embeddings beforehand (all-MiniLM-L6-v2, reduced from 384 to 32 dimensions with PCA and normalised to unit length). Each worker holds posts from two newsgroups and sends 200 rows downstream to a single sink worker B. A row is only `WORKER_ID, TS` and 32 `FLOAT32` values (144 bytes); the text itself never enters the topology.

B runs one SQL expression on every incoming row: `SCORE` = cosine similarity between the embedding and a fixed reference vector, the centroid of all *sci.space* posts. The edge workers are configured with `max_operators: 0`, so NebulaStream places this computation on B, not at the edge (verified in the query plan). B writes `WORKER_ID, TS, SCORE` and the embedding to a CSV file, which is the only input of both graphs below.

2. Did everything arrive?

B's output was compared row by row with what each worker sent:

Rows received at B	2,000 of 2,000 (every worker 200/200)
Embeddings at B vs. as sent	bit-identical (max difference 0)
SCORE at B vs. same formula in Python	max difference 5.7e-07 (float32 rounding)
Bytes sent as embeddings	281 KB
Same posts as raw text	2,356 KB → 8.4× more than the embeddings

3. What B received, coloured by its score

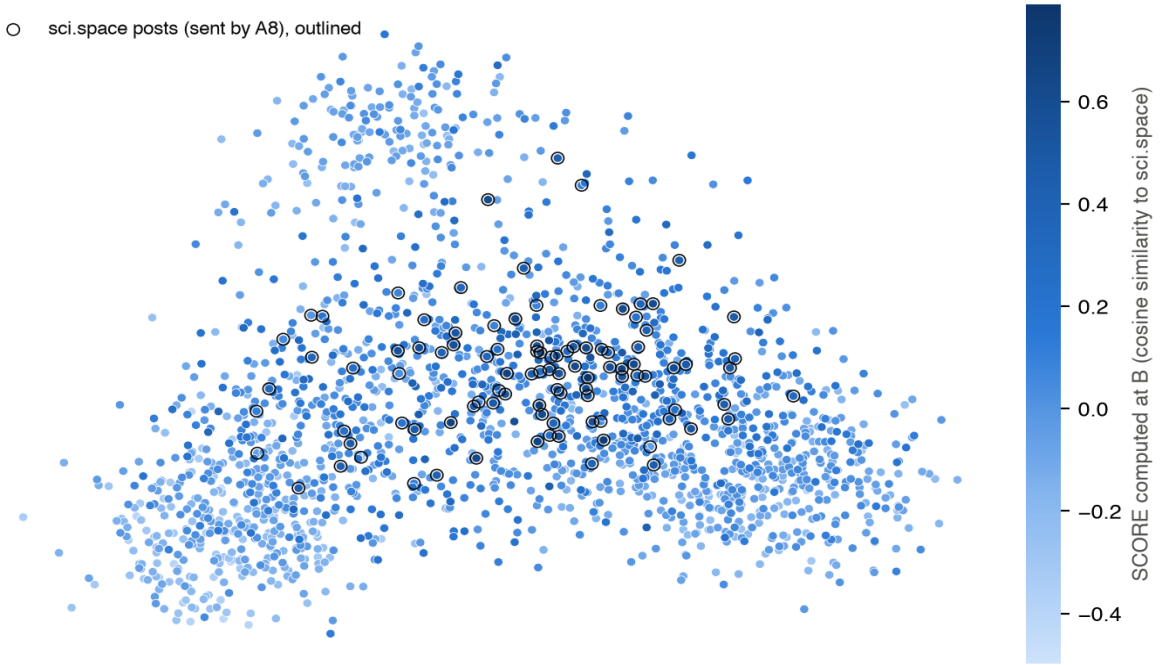


Figure 1. Every point is one of the 2,000 rows B received. The 32-dimensional embeddings are projected to two dimensions with PCA (the axes have no units; only distances matter). Colour is the SCORE that B computed in NebulaStream. Outlined points are the *sci.space* posts; this label comes from the dataset and was never sent to B.

The darkest points, i.e. the rows B scored as most similar to *sci.space*, are not scattered randomly: they are concentrated in one part of the plot, and the outlines show that they are mostly the *sci.space* posts sent by worker A8. The sci.space rows average a score of **+0.40**, while the next-highest of the other 19 newsgroups averages only +0.10. Of the 100 sci.space rows, 81 are among the 200 highest-scoring rows (top 10 %). B never saw any text, only 32 numbers per post, yet its computation recognises the topic. The embeddings therefore kept their meaning after travelling from the edge nodes through the topology to B.

4. Each node's data stays recognisable

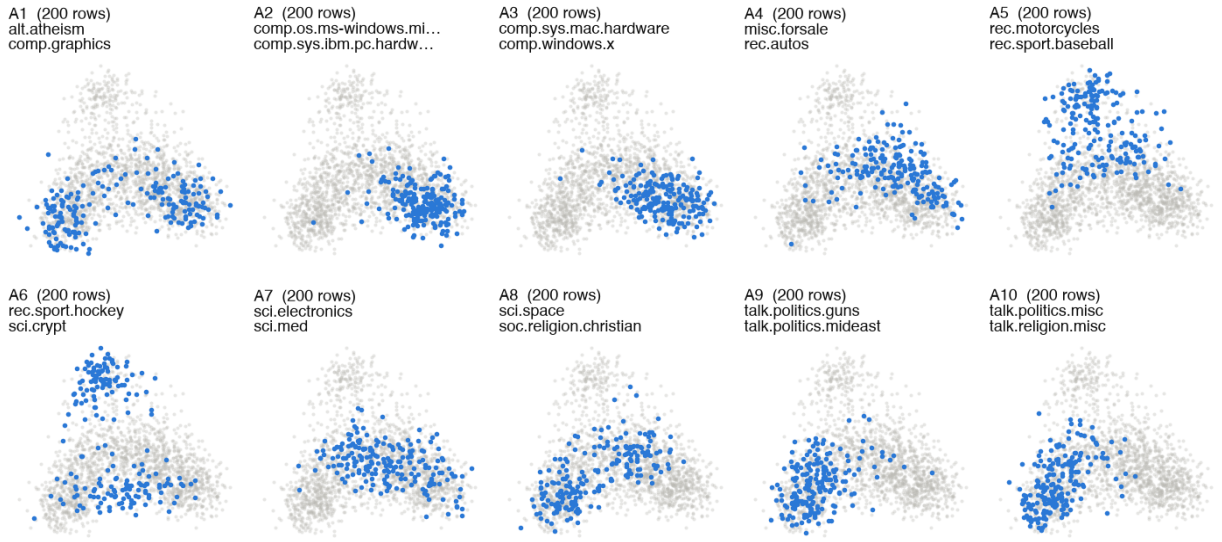


Figure 2. The same projection as Figure 1, drawn once per edge worker: that worker's rows in blue, all other rows in grey. Titles give the worker, the number of its rows that reached B, and its two newsgroups.

Each panel shows one node's data arriving at B in its own part of the plot (sometimes two parts, one per newsgroup, e.g. A5: motorcycles vs. baseball), distinct from most other workers. Workers whose topics are related overlap, as expected: A2 and A3 both hold computer groups, A9 and A10 both hold politics groups, and A8 and A10 share a religion group (these are also the three most similar pairs of worker centroids in the full 32-dimensional space). Workers with unrelated topics (e.g. A4: for-sale and cars vs. A8: space and religion) occupy different areas. This is what one would expect from meaningful embeddings, and it is visible after transport: all 10 streams were merged by NebulaStream at B, yet the structure of every node's data is preserved and every node delivered all 200 of its rows.

5. Conclusion

The demo shows that NebulaStream can carry embeddings from 10 edge nodes to one node: all 2,000 rows arrived complete and unchanged, B computed on them inside NebulaStream, and the result is meaningful (Figure 1: topic recognised by similarity; Figure 2: each node's data keeps its own structure). Sending embeddings instead of the raw posts needed 8.4× fewer bytes.

Limits of this first version. Embeddings were computed offline, not on the edge nodes, so the cost of computing them at the edge is not yet included. The byte comparison uses uncompressed text and payload bytes, not NebulaStream's wire format. A single SQL dot product compiles only up to 64 dimensions in this NebulaStream build (embeddings themselves travel fine up to 256). There is no aggregation yet; intermediate aggregating nodes are the next step toward hierarchical topologies.